

1 Rezolvare: Birocrație

1.1 Descrierea problemei

Problema ne cere să găsim un traseu de la colțul stânga-sus $(1, 1)$ până la colțul dreapta-jos (n, n) într-o matrice $n \times n$, cu condiția că din fiecare celulă putem merge doar la dreapta sau în jos. În plus, tronsoanele de drum pot fi de trei tipuri, fiecare cu restricții proprii de deplasare.

1.2 Observații cheie

Principală provocare constă în faptul că avem trei tipuri diferite de segmente:

- **Tip 0 (dp0):** deplasare orizontală sau verticală standard, permitând mișcarea $(i - 1, j)$ sau $(i, j - 1)$.
- **Tip 1 (dp1):** deplasare în diagonală negativă, permitând mutarea $(i - 1, j + 1)$ sau $(i, j - 1)$.
- **Tip 2 (dp2):** deplasare în diagonală pozitivă, permitând mutarea $(i + 1, j - 1)$ sau $(i, j + 1)$.

Observăm că toate drumurile pot fi construite prin combinarea acestor trei tipuri de segmente, iar starea finală din (n, n) poate fi atinsă din oricare dintre cele trei tipuri de DP.

1.3 Deducerea recurențelor

Pentru fiecare celulă (i, j) calculăm trei valori:

$$\begin{aligned} dp0[i][j] &= \max(dp[i - 1][j], dp[i][j - 1]) + m[i][j] \\ dp1[i][j] &= \max(dp1[i - 1][j + 1], dp0[i - 1][j + 1]) + m[i][j] \\ dp2[i][j] &= \max(dp2[i + 1][j - 1], dp0[i + 1][j - 1]) + m[i][j] \end{aligned}$$

Cea mai importantă observație este că trebuie să procesăm celulele în ordinea anti-diagonalelor (suma $i + j$ constantă), deoarece tranzițiile diagonale necesită informații de la celule deja calculate.

1.4 Algoritmul

Parcurgem toate anti-diagonalele de la 3 la $2n$. Pentru fiecare celulă de pe anti-diagonala curentă:

1. Calculăm $dp0$ din vecinii standard $(i - 1, j)$ și $(i, j - 1)$.
2. Calculăm $dp1$ din celula $(i - 1, j + 1)$ care are suma $i + j$ cu 1 mai mică.
3. Calculăm $dp2$ de la dreapta spre stânga, folosind $(i + 1, j - 1)$.
4. actualizăm $dp[i][j]$ cu maximul dintre toate cele trei stări.

Complexitatea este $O(n^2)$ atât în timp cât și în memorie.

1.5 Corectitudinea soluției

Demonstrația prin inducție pe anti-diagonale arată că după procesarea unei anti-diagonale, toate valorile dp pentru celulele de pe acea anti-diagonală sunt calculate corect. Baza este $(1, 1)$ unde toate stările sunt egale cu $m[1][1]$. Pasul inductiv folosește faptul că toate celulele necesare pentru tranziții au sume $i + j$ mai mici, deci au fost deja procesate.

1.6 Implementare

Codul sursă

```
#include <bits/stdc++.h>
#define INF 1000000001
#define L 1005
using namespace std;
ifstream fin("birocratie.in");
ofstream fout("birocratie.out");

int n;
int m[L][L], dp[L][L], dp0[L][L], dp1[L][L], dp2[L][L];

void board() {
    for (int i = 0; i < n + 2; i++)
        for (int j = 0; j < n + 2; j++)
            m[i][j] = dp[i][j] = dp0[i][j] = dp1[i][j] = dp2[i][j]
                = -INF;
}

int main() {
    fin >> n;
    board();
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            fin >> m[i][j];
    dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = m[1][1];
    for (int antidiag = 3; antidiag <= 2 * n; antidiag++) {
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n)
                continue;
            dp0[i][j] = max({dp0[i][j], dp[i - 1][j] + m[i][j], dp
                [i][j - 1] + m[i][j]});
        }
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n)
                continue;
        }
    }
}
```

```

        dp1[i][j] = max({dp1[i][j], dp1[i - 1][j + 1] + m[i][j]
                        }, dp0[i - 1][j + 1] + m[i][j]));
    }
    for (int i = n; i > 0; i--) {
        int j = antidiag - i;
        if (j < 1 || j > n)
            continue;
        dp2[i][j] = max({dp2[i][j], dp2[i + 1][j - 1] + m[i][j]
                        }, dp0[i + 1][j - 1] + m[i][j]));
    }
    for (int i = 1; i <= n; i++) {
        int j = antidiag - i;
        if (j < 1 || j > n)
            continue;
        dp[i][j] = max({dp[i][j], dp0[i][j], dp1[i][j], dp2[i]
                        [j]});
    }
}
fout << dp[n][n] << "\n";
return 0;
}

```

1.7 Note de implementare

Inițializarea cu $-INF$ este esențială pentru a evita depășirea valorilor din celulele neinitializate. Anti-diagonalele încep de la 3 deoarece primele două sunt doar $(1, 1)$ și $(1, 2)$ respectiv $(2, 1)$. Bucula pentru $dp2$ merge de la n la 1 pentru a asigura că informațiile necesare sunt disponibile în ordinea corectă.

Răspunsul final este $dp[n][n]$, reprezentând suma maximă posibilă a drumului de la $(1, 1)$ la (n, n) .